Advance Natural Language Processing (AML23702)

Lab Exercise - CIA: Program Codes
Department of Artificial Intelligence and Machine Learning

Question 1

Develop a Python program using spaCy to perform POS tagging and named entity recognition (NER) on a given paragraph. Display the identified words, their POS tags, and named entities.

Code (PGM1.py)

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = input("Enter a sentence: ")
doc = nlp(text)

print("POS Tagging:")
for token in doc:
    print(token.text, token.pos_)

print("\nNamed Entities:")
for ent in doc.ents:
    print(ent.text, ent.label_)
```

Question 2

Develop a simple information retrieval-based Question Answering (QA) system that accepts a question and retrieves the most relevant sentence from a given set of documents.

Code (PGM2.py)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = [
    "Python is a programming language.",
    "Machine learning uses data to learn.",
    "NLP processes human language.",
]

question = input("Enter your question: ")

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs + [question])

scores = cosine_similarity(X[-1], X[:-1])

print("Answer:", docs[scores.argmax()])
```

Question 4

Apply Latent Dirichlet Allocation (LDA) to a small collection of text documents and display the top five words associated with each discovered topic.

Code (PGM4.py)

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

docs = [
    "Python and Java are programming languages",
    "Python is used for machine learning",
    "Football and cricket are popular sports",
    "Cricket is played with bat and ball"
]

v = CountVectorizer(stop_words="english")
X = v.fit_transform(docs)

lda = LatentDirichletAllocation(n_components=2, random_state=0)
lda.fit(X)

words = v.get_feature_names_out()

for i, topic in enumerate(lda.components_):
    top = topic.argsort()[-5:][::-1]
    print("Topic", i + 1, ":", [words[j] for j in top])
```

Question 5

Write a Python program using NLTK WordNet to identify the synonyms and definitions of a given word and distinguish its possible meanings in different contexts.

Code (PGM5.py)

```
import nltk
from nltk.corpus import wordnet as wn

nltk.download('wordnet')

word = input("Enter a word: ")

for s in wn.synsets(word):
    print("\nMeaning:", s.definition())
    print("Synonyms:", [l.name() for l in s.lemmas()])
```